Name: Soham Chavan

PRN: 124B1F022

Class: IT A

Assignment No. 4

Aim:

To implement Dijkstra's algorithm to find the shortest path from the ambulance's source location to the nearest hospital in a city road network, and to account for dynamic updates in road travel time due to changing traffic conditions.

Code:

```
import java.util.*;

class Edge {
    int destination;
    int weight;

    Edge(int destination, int weight) {
        this.destination = destination;
        this.weight = weight;
    }
}

public class DijkstraAmbulance {

    static Map<Integer, List<Edge>> graph = new HashMap<>();

    static void addEdge(int u, int v, int weight) {
        graph.computeIfAbsent(u, k -> new ArrayList<>()).add(new Edge(v, weight));
        graph.computeIfAbsent(v, k -> new ArrayList<>()).add(new Edge(u, weight));
    }

    static void updateTraffic(int u, int v, int newWeight) {
        for (Edge e : graph.get(u)) if (e.destination == v) e.weight = newWeight;
        for (Edge e : graph.get(v)) if (e.destination == u) e.weight = newWeight;
    }

    static int[] dijkstra(int source, int numNodes, int[] parent) {
        int[] dist = new int[numNodes];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[source] = 0;
        Arrays.fill(parent, -1);

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.add(new int[]{source, 0});
        boolean[] visited = new boolean[numNodes];

        while (!pq.isEmpty()) {
```

```
            int[] current = pq.poll();
            int u = current[0];
            if (visited[u]) continue;
            visited[u] = true;

            for (Edge e : graph.getOrDefault(u, new ArrayList<>())) {
                if (!visited[e.destination] && dist[u] + e.weight < dist[e.destination]) {
                    dist[e.destination] = dist[u] + e.weight;
                    parent[e.destination] = u;
                    pq.add(new int[]{e.destination, dist[e.destination]});
                }
            }
        }
        return dist;
    }

    static void printPath(int node, int[] parent, StringBuilder sb) {
        if (node == -1) return;
        if (parent[node] != -1) {
            printPath(parent[node], parent, sb);
            sb.append(" -> ");
        }
        sb.append(node);
    }

    public static void main(String[] args) {
        int numNodes = 7;
        // 0 = ambulance source, 5 and 6 = hospitals
        addEdge(0, 1, 4);
        addEdge(0, 2, 2);
        addEdge(1, 2, 1);
        addEdge(1, 3, 5);
        addEdge(2, 3, 8);
        addEdge(2, 4, 10);
        addEdge(3, 4, 2);
        addEdge(3, 5, 6);
        addEdge(4, 5, 3);
        addEdge(4, 6, 7);
        addEdge(5, 6, 1);

        int source = 0;
        int[] parent = new int[numNodes];
        int[] dist = dijkstra(source, numNodes, parent);

        System.out.println("Shortest travel time from Ambulance (Node " + source + ") to all
junctions:\n");
        for (int i = 0; i < numNodes; i++) {
            System.out.println("Node " + i + " : " + dist[i] + " minutes");
        }

        int[] hospitals = {5, 6};
        int nearestHospital = -1;
        int minTime = Integer.MAX_VALUE;
        for (int h : hospitals) {
            if (dist[h] < minTime) {
                minTime = dist[h];
                nearestHospital = h;
            }
        }
```

```
        StringBuilder path = new StringBuilder();
        printPath(nearestHospital, parent, path);

        System.out.println("\nNearest Hospital: Node " + nearestHospital);
        System.out.println("Minimum Travel Time: " + minTime + " minutes");
        System.out.println("Optimal Route: " + path);

        System.out.println("\n--- Simulating real-time traffic congestion update ---");
        System.out.println("Updating road (0 -> 2) travel time from 2 to 15 minutes due to
congestion...");
        updateTraffic(0, 2, 15);

        int[] parent2 = new int[numNodes];
        int[] dist2 = dijkstra(source, numNodes, parent2);

        int nearestHospital2 = -1;
        int minTime2 = Integer.MAX_VALUE;
        for (int h : hospitals) {
            if (dist2[h] < minTime2) {
                minTime2 = dist2[h];
                nearestHospital2 = h;
            }
        }

        StringBuilder path2 = new StringBuilder();
        printPath(nearestHospital2, parent2, path2);

        System.out.println("\nUpdated Nearest Hospital: Node " + nearestHospital2);
        System.out.println("Updated Minimum Travel Time: " + minTime2 + " minutes");
        System.out.println("Updated Optimal Route: " + path2);
    }
}
```

Output:

```
Shortest travel time from Ambulance (Node 0) to all junctions:

Node 0 : 0 minutes
Node 1 : 3 minutes
Node 2 : 2 minutes
Node 3 : 8 minutes
Node 4 : 10 minutes
Node 5 : 13 minutes
Node 6 : 14 minutes

Nearest Hospital: Node 5
Minimum Travel Time: 13 minutes
Optimal Route: 0 -> 2 -> 1 -> 3 -> 4 -> 5

--- Simulating real-time traffic congestion update ---
Updating road (0 -> 2) travel time from 2 to 15 minutes due to congestion...

Updated Nearest Hospital: Node 5
Updated Minimum Travel Time: 14 minutes
Updated Optimal Route: 0 -> 1 -> 3 -> 4 -> 5
```

Conclusion:

Dijkstra's algorithm was implemented using a priority queue (min-heap) to efficiently compute the shortest travel time from the ambulance's source node to every junction in the road network, with a time complexity of O((V+E) log V). The algorithm was also tested with a dynamic traffic update, where increasing the weight of one road segment caused the algorithm to correctly recompute a new optimal route to the nearest hospital, demonstrating its usefulness for real-time emergency navigation in a smart city.